

CISSP Manager-Mindset Guide

DOMAIN 3 | SECURITY ARCHITECTURE AND ENGINEERING

Design principles • Models • Platforms • Cryptography • Facilities • Lifecycle

PURPOSE A decision-oriented guide to translating business and security requirements into trustworthy architectures, controls, cryptographic choices, facilities, and lifecycle decisions.

Aligned to the ISC2 CISSP Certification Exam Outline effective April 15, 2024
Domain weight: 13%

How to Use This Guide

Domain 3 contains substantial technical material, but the exam still evaluates judgment. Know what technologies do, then select them only after the business objective, security requirement, threat model, trust boundary, lifecycle, and operating constraints are understood.

DEFAULT DECISION LENS Requirements before architecture; architecture before products; secure design before compensating controls; verified lifecycle governance before operational trust.

The Domain 3 Manager Mindset

- Start with stakeholder needs, business impact, legal obligations, threat models, and security requirements.
- Prefer simple, layered designs with secure defaults, least privilege, isolation, and explicit trust decisions.
- A security model expresses policy rules; an architecture realizes them; controls implement them; testing verifies them.
- Do not select cryptography by algorithm popularity alone—consider purpose, key lifecycle, implementation, performance, interoperability, and longevity.
- Availability, safety, maintainability, supportability, and recoverability are security requirements.
- Physical and environmental failures can defeat logical controls.
- Assurance requires evidence that design and implementation satisfy requirements—not confidence in a brand or certification label.
- Retirement and disposal are designed lifecycle stages, not last-minute cleanup.

Domain 3 Roadmap

Objective	Focus	Primary decision question
3.1	Secure design principles	Which principles reduce risk at the design level?
3.2	Security models	Which formal policy is the model designed to enforce?
3.3	Control selection	Which controls satisfy defined system requirements?
3.4	Information-system capabilities	What trust, isolation, memory, and hardware features are available?
3.5	Architecture vulnerabilities	How does the platform or deployment model change risk?
3.6	Cryptographic solutions	What cryptographic objective and lifecycle are required?
3.7	Cryptanalytic attacks	What is being attacked: math, protocol, implementation, key, or endpoint?
3.8–3.9	Site and facility security	How do location, environment, utilities, and physical access affect risk?
3.10	System lifecycle	How is security governed from need through retirement?

3.1 Secure Design Principles

Least Privilege and Need-to-Know

Easy definition: Least privilege grants only the minimum permissions needed; need-to-know restricts access to information required for an authorized task.

When it matters: Accounts, services, administrators, applications, data, network paths, cloud roles, and temporary elevation.

THINK LIKE A MANAGER Design small privileges, short duration, explicit approval, separation of duties, and review; broad standing access is operational debt.

EXAM TRAPS

- Equating successful authentication with authorization.
- Granting permissions for convenience or hypothetical future need.
- Applying least privilege only to human users and ignoring services, APIs, agents, and devices.
- Failing to remove privileges after a role or task ends.

QUICK DISTINCTION Least privilege limits capability; need-to-know limits information; both are required even for trusted users.

Defense in Depth

Easy definition: Defense in depth uses independent, complementary layers so one failure does not expose the asset or mission.

When it matters: Architecture, network segmentation, identity, application security, monitoring, recovery, and physical security.

THINK LIKE A MANAGER Layer controls across prevention, detection, response, and recovery, and avoid layers that share one hidden failure mode.

EXAM TRAPS

- Counting multiple products with the same dependency as independent layers.
- Using layers to excuse a fundamentally insecure design.
- Choosing only preventive controls.
- Assuming more controls always reduce risk; complexity can create gaps and failure.

QUICK DISTINCTION Defense in depth is layered risk reduction; redundancy duplicates capability for availability; diversity reduces common-mode failure.

Secure Defaults, Fail Securely, and Complete Mediation

Easy definition: Secure defaults deny or minimize access until explicitly authorized; fail-secure behavior preserves protection during failure; complete mediation checks every access rather than relying on stale trust.

When it matters: Initialization, error handling, outages, authorization decisions, firewalls, applications, and safety-related systems.

THINK LIKE A MANAGER Define the safe state from business and safety requirements, then test failures—not

merely normal operation.

EXAM TRAPS

- Assuming fail secure always means fail closed; life safety or availability can change the safe state.
- Using permissive defaults to simplify deployment.
- Checking authorization only at login.
- Ignoring cached credentials and bypass modes during dependency failure.

QUICK DISTINCTION Fail secure protects the security objective during failure; fail safe prioritizes safety. Context determines whether access should open or close.

Separation of Duties and Privilege Separation

Easy definition: Separation of duties divides sensitive processes among distinct roles; privilege separation divides a system into components with only the permissions each requires.

When it matters: Transactions, changes, code deployment, key management, administration, and software architecture.

THINK LIKE A MANAGER Prevent one person or component from authorizing, executing, and concealing a critical action.

EXAM TRAPS

- Using two accounts controlled by the same person as separation.
- Confusing dual control with two-person review after the fact.
- Creating separation on paper while shared credentials bypass it.
- Ignoring emergency-access monitoring and reconciliation.

QUICK DISTINCTION Separation of duties divides responsibility; dual control requires two parties together; split knowledge divides a secret so no one holds it all.

Keep It Simple, Economy of Mechanism, and Least Common Mechanism

Easy definition: Simple, small mechanisms are easier to understand and verify; minimizing shared mechanisms reduces unintended information paths and common failures.

When it matters: Architecture, authentication services, shared infrastructure, security code, dependencies, and multitenancy.

THINK LIKE A MANAGER Prefer the least complex design that meets requirements and isolate high-risk functions where shared mechanisms increase blast radius.

EXAM TRAPS

- Confusing simplicity with lack of defense.
- Adding tools instead of fixing unclear trust boundaries.
- Centralizing every service without considering concentration risk.
- Allowing convenience features to create hidden paths around controls.

QUICK DISTINCTION Centralization can improve consistency; isolation can reduce shared failure. Choose based on risk, resilience, and manageability.

Zero Trust and Trust but Verify

Easy definition: Zero trust continuously evaluates explicit access using identity, device, resource, context, and policy rather than granting broad trust from network location.

When it matters: Hybrid work, cloud, third parties, microsegmentation, privileged access, and service-to-service communication.

THINK LIKE A MANAGER Protect resources with explicit, least-privilege decisions and continuous signals; implement incrementally around defined business flows and assets.

EXAM TRAPS

- Treating zero trust as a product.
- Assuming internal networks become irrelevant.
- Requiring repeated authentication without improving authorization or context.
- Ignoring availability and recovery of policy, identity, and telemetry services.

QUICK DISTINCTION Zero trust removes implicit trust; it does not claim no trust exists—authorized access still depends on verified assertions and policy.

Privacy by Design and Shared Responsibility

Easy definition: Privacy by design embeds minimization, purpose, transparency, user rights, and protection into architecture; shared responsibility allocates security duties among customer, provider, and other parties.

When it matters: Cloud, SaaS, AI, analytics, personal data, managed services, and system acquisition.

THINK LIKE A MANAGER Map responsibilities and privacy requirements to named owners and evidence; unassigned responsibility becomes a control gap.

EXAM TRAPS

- Adding privacy controls only after deployment.
- Assuming the provider secures customer configurations, identities, and data by default.
- Confusing responsibility allocation with transfer of accountability.
- Using a generic shared-responsibility diagram without service-specific details.

QUICK DISTINCTION The provider and customer have different duties, but the organization remains accountable for its risk and obligations.

SASE and Secure Service Edge

Easy definition: Secure Access Service Edge combines network and security capabilities delivered largely as cloud services; SSE focuses on security services without the full WAN component.

When it matters: Distributed users, branches, SaaS, remote access, SD-WAN, web gateways, CASB, and zero-trust access.

THINK LIKE A MANAGER Start with traffic, identity, data, resilience, jurisdiction, and service requirements; architecture matters more than buying a bundled label.

EXAM TRAPS

- Treating SASE as one appliance or zero trust itself.
- Ignoring provider concentration and internet dependency.
- Backhauling traffic unnecessarily without latency analysis.
- Assuming all components have equal integration and policy maturity.

QUICK DISTINCTION SASE combines networking plus security services; SSE is the security subset, commonly including SWG, CASB, and ZTNA.

3.2 Security Models

Security Models and Reference Monitors

Easy definition: A security model formally expresses rules for protecting information; a reference monitor is the abstract mechanism that mediates access and should be tamper-resistant, always invoked, and small enough to verify.

When it matters: Evaluating access-control architectures, trusted computing bases, confidentiality, integrity, and assurance.

THINK LIKE A MANAGER Choose the model that matches the policy objective; do not select a confidentiality model to solve an integrity problem.

EXAM TRAPS

- Confusing a model with an implementation or product.
- Memorizing rules without identifying the protected property.
- Assuming a model addresses availability.
- Calling any access-control check a fully trustworthy reference monitor.

QUICK DISTINCTION Model = formal policy rules; reference monitor = mediation concept; security kernel = trusted implementation; TCB = all trusted components enforcing policy.

Bell–LaPadula

Easy definition: Bell–LaPadula is a confidentiality model that prevents information from flowing from higher classifications to lower ones.

When it matters: Multilevel confidentiality and government-style classified environments.

THINK LIKE A MANAGER Use it when unauthorized disclosure is the primary concern and pair it with separate integrity and availability controls.

EXAM TRAPS

- Choosing it for integrity.
- Reversing “no read up” and “no write down.”
- Assuming discretionary controls alone enforce labels.
- Forgetting trusted subjects or declassification require controlled exceptions.

QUICK DISTINCTION Simple security property: no read up. Star property: no write down. Goal: confidentiality.

Biba

Easy definition: Biba is an integrity model designed to prevent lower-integrity subjects or data from contaminating higher-integrity information.

When it matters: Trusted data processing, critical records, and integrity-dominant systems.

THINK LIKE A MANAGER Use integrity levels and controlled flows when trustworthy input and modification matter more than secrecy.

EXAM TRAPS

- Reversing Biba with Bell–LaPadula.
- Assuming high confidentiality means high integrity.
- Ignoring availability and authenticity.
- Applying strict Biba where business workflows require controlled lower-level input without mediation.

QUICK DISTINCTION Simple integrity: no read down. Star integrity: no write up. Goal: integrity—the conceptual inverse of Bell–LaPadula.

Clark–Wilson and Brewer–Nash

Easy definition: Clark–Wilson protects commercial integrity through well-formed transactions, separation of duties, and constrained programs; Brewer–Nash prevents conflicts of interest by dynamically restricting access after a user enters a competing dataset.

When it matters: Financial systems, business transactions, auditability, consulting, legal, and market-sensitive information.

THINK LIKE A MANAGER Select Clark–Wilson for controlled business transactions and Brewer–Nash for conflict-of-interest boundaries.

EXAM TRAPS

- Treating Clark–Wilson as a lattice confidentiality model.
- Ignoring separation of duties and integrity verification procedures.
- Assuming Brewer–Nash restrictions are static before any access occurs.
- Confusing conflict-of-interest classes with classification levels.

QUICK DISTINCTION Clark–Wilson = commercial integrity. Brewer–Nash/Chinese Wall = dynamic confidentiality based on prior access and conflicts.

Information-Flow and Noninterference Models

Easy definition: Information-flow models control permitted movement between security levels; noninterference seeks to ensure actions at a high level do not influence what lower-level users can observe.

When it matters: Covert channels, multilevel systems, privacy isolation, and high-assurance design.

THINK LIKE A MANAGER Analyze actual flows and observable effects—not only access-control lists—and reduce shared mechanisms that leak information.

EXAM TRAPS

- Assuming denied direct access prevents indirect flow.
- Ignoring timing, resource usage, error messages, and metadata.
- Treating every side effect as practically eliminable.
- Confusing covert channels with ordinary authorized communication.

QUICK DISTINCTION Access control governs permitted operations; information-flow analysis asks where information can actually travel or be inferred.

3.3 Select Controls from Security Requirements

Requirements-Driven Control Selection

Easy definition: Security requirements translate stakeholder needs, threats, obligations, and risk into measurable properties the system must satisfy.

When it matters: Architecture, procurement, engineering, accreditation, exceptions, and verification.

THINK LIKE A MANAGER Define what must be protected and how success will be measured before comparing controls or vendors.

EXAM TRAPS

- Beginning with a preferred technology.
- Writing vague requirements such as “use strong encryption.”
- Confusing a design choice with the underlying requirement.
- Failing to trace requirements to tests, owners, and residual risk.

QUICK DISTINCTION Requirement states the needed outcome; control provides protection; implementation realizes the control; verification provides evidence.

Control Types, Assurance, and Trust

Easy definition: Controls may be managerial, operational, technical, or physical; assurance is justified confidence based on evidence that controls are correctly designed and effective.

When it matters: System authorization, architecture reviews, certifications, evaluation, testing, and supply-chain decisions.

THINK LIKE A MANAGER Seek evidence proportional to risk and understand the scope, assumptions, and date of every assurance claim.

EXAM TRAPS

- Equating trust with assurance.
- Assuming certification covers all configurations and uses.
- Relying on vendor statements without evidence.
- Testing only implementation while ignoring design adequacy.

QUICK DISTINCTION Trust is reliance; assurance is the evidence-based confidence that reliance is justified.

3.4 Security Capabilities of Information Systems

Trusted Computing Base and Protection Rings

Easy definition: The TCB includes hardware, software, and firmware relied on to enforce security; protection rings separate privilege levels so the most trusted code has the greatest control.

When it matters: Operating systems, hypervisors, kernels, drivers, and high-assurance designs.

THINK LIKE A MANAGER Minimize and protect the TCB, reduce privileged code, and verify boundaries; compromise of the TCB can invalidate higher-layer controls.

EXAM TRAPS

- Calling the entire system equally trusted.
- Assuming ring numbers are identical across all architectures.
- Believing user-mode isolation prevents kernel compromise.
- Ignoring firmware and management interfaces in the TCB.

QUICK DISTINCTION Ring 0 is typically most privileged; outer rings are less privileged. The TCB is broader than the security kernel.

Memory Protection and Process Isolation

Easy definition: Memory protection prevents one process from improperly reading or modifying another process or the operating system through address-space isolation and execution protections.

When it matters: Operating systems, applications, virtualization, malware resistance, and exploitation mitigation.

THINK LIKE A MANAGER Use layered isolation and secure coding; memory controls reduce exploitability but do not correct vulnerable logic.

EXAM TRAPS

- Assuming DEP/NX prevents all code execution attacks.
- Confusing ASLR with encryption.
- Ignoring shared memory, side channels, DMA, and privileged drivers.
- Treating containers as hardware-level isolation.

QUICK DISTINCTION DEP/NX limits execution from data regions; ASLR randomizes locations; bounds checking helps stop improper memory access.

TPM, HSM, Secure Boot, and Attestation

Easy definition: A TPM is a platform security chip for protected keys and measurements; an HSM is specialized key-management and cryptographic hardware; secure boot verifies trusted startup components; attestation reports measured state.

When it matters: Device identity, disk encryption, key protection, code integrity, PKI, and trusted startup.

THINK LIKE A MANAGER Match the trust anchor and assurance level to the threat and operational need; protect recovery and administrative paths.

EXAM TRAPS

- Treating TPM and HSM as interchangeable.
- Assuming secure boot proves the running system remains uncompromised.
- Storing keys securely while exposing them through weak applications or administrators.
- Ignoring backup, rotation, high availability, and destruction of protected keys.

QUICK DISTINCTION TPM is usually bound to one platform; HSM serves higher-assurance centralized or application cryptography; attestation provides evidence of measured state.

3.5 Architecture and Solution Vulnerabilities

Client, Server, and Database Systems

Easy definition: Clients face untrusted users and content; servers concentrate services and privileges; databases concentrate valuable structured data and powerful query paths.

When it matters: Hardening, segmentation, endpoint security, service design, database permissions, monitoring, and patching.

THINK LIKE A MANAGER Reduce exposed functionality, separate duties and tiers, protect administrative paths, and prioritize controls by business impact and attack surface.

EXAM TRAPS

- Assuming a trusted internal client is safe.
- Running unnecessary services or shared administrator accounts.
- Granting applications database-owner privileges.
- Protecting database files but ignoring queries, exports, backups, logs, and inference.

QUICK DISTINCTION Client risk often includes hostile input and local users; server risk includes exposed services; database risk includes aggregation, privilege, and query abuse.

Industrial Control and Embedded Systems

Easy definition: ICS and embedded systems interact with physical processes and often have safety, availability, real-time, longevity, and vendor constraints unlike typical enterprise IT.

When it matters: Power, manufacturing, transportation, building systems, medical devices, IoT, and appliances.

THINK LIKE A MANAGER Prioritize life safety and reliable operation, understand the process, coordinate with engineering and operations, test changes, and use compensating controls where patching or replacement is constrained.

EXAM TRAPS

- Applying enterprise patch urgency without operational testing.
- Active scanning or testing that could disrupt fragile devices.
- Assuming air gaps eliminate removable media, vendor, wireless, or maintenance paths.
- Using confidentiality as the dominant objective when safety and availability drive risk.

QUICK DISTINCTION OT security protects a physical process; consequence-based risk and safe change often outweigh conventional IT speed.

Cloud and Shared Responsibility

Easy definition: Cloud service models shift—but do not eliminate—customer responsibilities for data, identity, configuration, applications, workloads, and governance.

When it matters: SaaS, PaaS, IaaS, multitenancy, regions, provider dependencies, and cloud migration.

THINK LIKE A MANAGER Select the service model from business requirements, map every responsibility, validate provider evidence, design resilience and exit, and continuously monitor customer configuration.

EXAM TRAPS

- Assuming the provider secures everything.
- Treating compliance reports as customer authorization.
- Ignoring control-plane identities and APIs.
- Designing availability in one provider region without checking failure domains.
- Failing to plan data portability and exit.

QUICK DISTINCTION As service abstraction increases from IaaS to PaaS to SaaS, provider operational responsibility grows, but customer accountability for use, identity, data, and configuration remains.

Virtualization and Containers

Easy definition: Virtualization abstracts hardware into virtual machines under a hypervisor; containers share a host kernel while isolating application processes and dependencies.

When it matters: Consolidation, cloud, workload isolation, images, orchestration, snapshots, and rapid deployment.

THINK LIKE A MANAGER Protect the management plane, harden hosts and images, minimize privileges, separate high-risk workloads, and govern rapid lifecycle and sprawl.

EXAM TRAPS

- Equating container isolation with a separate VM.
- Ignoring hypervisor or orchestration compromise as a concentration risk.
- Treating snapshots as backups.
- Running privileged containers or untrusted images.
- Failing to patch templates and redeploy immutable workloads.

QUICK DISTINCTION VMs virtualize hardware and usually have separate kernels; containers share the host kernel and provide lighter isolation.

Microservices, APIs, and Serverless

Easy definition: Microservices decompose applications into independently deployed services; APIs connect them; serverless runs functions on provider-managed infrastructure.

When it matters: Cloud-native systems, distributed trust, CI/CD, event processing, and third-party integrations.

THINK LIKE A MANAGER Define service identities, authorization, schemas, secrets, trust boundaries, observability, resilience, and supply-chain controls before scaling deployment.

EXAM TRAPS

- Trusting internal service calls automatically.
- Using an API gateway as the only authorization layer.
- Embedding secrets in code or environment artifacts.
- Ignoring event injection, dependency failure, excessive permissions, and provider limits.
- Assuming serverless means no servers or no patch responsibility at any layer.

QUICK DISTINCTION Microservices change system decomposition; serverless changes execution and operational abstraction; APIs remain security boundaries.

IoT, Edge, Distributed, and High-Performance Systems

Easy definition: These systems distribute processing across constrained devices, remote locations, clusters, or specialized high-speed environments, increasing trust, update, identity, physical, and coordination challenges.

When it matters: Sensors, gateways, 5G edge, research clusters, AI workloads, smart facilities, and geographically distributed systems.

THINK LIKE A MANAGER Inventory components, establish device and service identity, protect update paths, segment trust, plan offline operation, and design for physical exposure and fleet-scale recovery.

EXAM TRAPS

- Assuming small devices have low business impact.
- Ignoring insecure default credentials and unsupported firmware.
- Treating edge data as temporary and unclassified.
- Centralizing control without planning for lost connectivity.
- Applying ordinary enterprise controls without performance and latency analysis.

QUICK DISTINCTION Edge moves compute near data or operations; distributed systems coordinate across nodes; HPC optimizes tightly coupled computation—each creates different trust and availability concerns.

3.6 Cryptographic Solutions

Cryptographic Objectives and Selection

Easy definition: Cryptography can support confidentiality, integrity, authentication, and nonrepudiation, but each objective requires the correct primitive, protocol, keys, and implementation.

When it matters: Data protection, communications, identity, software signing, transactions, storage, and key exchange.

THINK LIKE A MANAGER State the objective and threat first; select approved, current algorithms and protocols with sufficient key strength, interoperability, performance, and lifecycle support.

EXAM TRAPS

- Choosing encryption when a hash or signature is needed.
- Inventing proprietary cryptography.
- Using strong algorithms in weak modes or protocols.
- Ignoring metadata, endpoints, and key compromise.
- Assuming cryptography guarantees availability.

QUICK DISTINCTION Encryption provides confidentiality; hashes detect change; MACs provide integrity/authenticity with a shared secret; digital signatures add public verification and nonrepudiation support.

Symmetric and Asymmetric Cryptography

Easy definition: Symmetric encryption uses a shared secret and is fast for bulk data; asymmetric cryptography uses a public/private key pair and supports key establishment, signatures, and scalable identity.

When it matters: TLS, VPNs, storage, messaging, PKI, key exchange, and signatures.

THINK LIKE A MANAGER Use hybrid designs: asymmetric mechanisms authenticate or establish secrets; symmetric keys protect bulk data efficiently.

EXAM TRAPS

- Assuming asymmetric encryption is always more secure.
- Encrypting large volumes directly with public-key algorithms.
- Believing a public key is trustworthy without validation.
- Confusing key size comparisons across different algorithm families.

QUICK DISTINCTION Symmetric = one shared secret and speed. Asymmetric = key pair, easier distribution, signatures, and higher computational cost.

Hashing, MACs, and Digital Signatures

Easy definition: A hash creates a fixed-size digest without a secret; a MAC combines data with a shared secret; a digital signature uses a private key and is verified with a public key.

When it matters: Integrity checks, password storage, message authentication, code signing, transactions, and evidence.

THINK LIKE A MANAGER Select the mechanism based on whether verification must be secret, shared, or public

and protect identities and keys accordingly.

EXAM TRAPS

- Calling hashing encryption.
- Using a plain hash to authenticate a message from an attacker.
- Claiming a shared-key MAC provides strong nonrepudiation.
- Storing passwords with fast unsalted hashes.
- Signing a hash without validating the signer's certificate and key status.

QUICK DISTINCTION Hash = integrity evidence only. MAC = integrity plus shared-secret authenticity. Signature = integrity, origin authentication, and nonrepudiation support.

PKI, Certificates, and Trust

Easy definition: PKI binds public keys to identities through certificates, registration, issuance, validation, revocation, policy, and trusted authorities.

When it matters: TLS, user and device identity, email, code signing, document signing, and machine-to-machine trust.

THINK LIKE A MANAGER Design the certificate and key lifecycle, validate the trust chain and intended use, monitor expiry and revocation, and protect issuing authorities.

EXAM TRAPS

- Assuming a valid certificate means the subject is safe.
- Checking expiration but not name, chain, usage, or revocation.
- Confusing a registration authority with a certification authority.
- Failing to protect root and issuing CA keys.
- Ignoring renewal outages and crypto agility.

QUICK DISTINCTION RA verifies identity; CA signs certificates; repository publishes certificates/status; relying party validates before trust.

Key Management and Cryptographic Lifecycle

Easy definition: Key management governs generation, distribution, storage, access, rotation, backup or escrow, recovery, revocation, archival, and destruction.

When it matters: Every encryption, signing, authentication, and PKI deployment.

THINK LIKE A MANAGER Treat keys as high-value assets, separate duties, automate lifecycle, maintain recoverability where appropriate, and destroy keys when access must end.

EXAM TRAPS

- Focusing on algorithm strength while storing keys with the ciphertext.
- Escrowing signing keys that should uniquely represent a signer.
- Rotating keys without planning re-encryption or historical validation.
- Destroying decryption keys before retention ends.
- Using one key for multiple purposes or excessive scope.

QUICK DISTINCTION Cryptoperiod is the approved key-use period; rotation replaces active keys; revocation invalidates trust; destruction can provide cryptographic erasure.

Elliptic-Curve, Quantum, and Crypto Agility

Easy definition: ECC provides strong asymmetric security with smaller keys; quantum computing threatens widely used public-key algorithms and affects long-lived data; crypto agility enables controlled algorithm and key changes.

When it matters: Mobile and constrained systems, certificates, long-term confidentiality, procurement, standards, and migration planning.

THINK LIKE A MANAGER Inventory cryptographic dependencies, protect data based on its secrecy lifetime, use approved post-quantum transition guidance, and design replaceable algorithms and protocols.

EXAM TRAPS

- Comparing ECC and RSA key sizes directly.
- Assuming present-day quantum computers can instantly break all encryption.
- Ignoring “harvest now, decrypt later” for long-lived secrets.
- Hard-coding algorithms and keys into systems.
- Adopting immature cryptography without standards and interoperability review.

QUICK DISTINCTION Quantum risk is strongest for current public-key methods; symmetric algorithms are generally addressed with adequate key sizes, while migration requires inventory and agility.

3.7 Cryptanalytic and Implementation Attacks

Brute Force and Classical Cryptanalysis

Easy definition: Brute force tries keys or credentials; ciphertext-only, known-plaintext, chosen-plaintext or ciphertext, and frequency analysis use available data and oracle access to exploit patterns or weaknesses.

When it matters: Algorithm selection, key size, protocol design, authentication, and understanding attack assumptions.

THINK LIKE A MANAGER Use approved algorithms, sufficient keys, rate limits where relevant, and modern modes; evaluate the attacker's actual information and capabilities.

EXAM TRAPS

- Calling password guessing cryptanalysis in every context.
- Assuming longer keys fix a broken algorithm or implementation.
- Using deterministic or pattern-revealing encryption.
- Ignoring weak random-number generation.

QUICK DISTINCTION Brute force exploits finite keyspace; cryptanalysis exploits structure, patterns, or oracle behavior.

Side Channels, Timing, and Fault Injection

Easy definition: Side-channel attacks infer secrets from timing, power, electromagnetic emissions, cache behavior, or other leakage; fault injection deliberately causes errors to reveal or bypass protection.

When it matters: Cryptographic implementations, smart cards, HSMS, shared processors, embedded devices, and physical adversaries.

THINK LIKE A MANAGER Evaluate implementation and environment, use constant-time and hardened designs, isolate sensitive workloads, and protect physical access.

EXAM TRAPS

- Assuming mathematically strong cryptography prevents implementation leakage.
- Ignoring shared-cloud or shared-cache observation.
- Treating timing differences as harmless performance details.
- Overlooking voltage, clock, laser, or environmental manipulation of hardware.

QUICK DISTINCTION Cryptanalysis may target mathematics; side channels and faults target how the system computes or behaves.

MITM, Pass-the-Hash, Kerberos Abuse, and Ransomware

Easy definition: These attacks exploit protocol trust, stolen authenticators, credential material, identity infrastructure, or operational dependencies rather than directly breaking cryptographic algorithms.

When it matters: Network authentication, Windows environments, privileged access, lateral movement, backup, and incident resilience.

THINK LIKE A MANAGER Protect identities and endpoints, validate peers, minimize privilege, segment

administrative paths, monitor abnormal use, and maintain tested isolated recovery.

EXAM TRAPS

- Classifying every attack near encryption as cryptanalysis.
- Assuming TLS prevents MITM without certificate validation.
- Believing pass-the-hash requires cracking the password.
- Protecting Kerberos tickets while leaving privileged endpoints exposed.
- Treating ransomware only as an availability event; theft and extortion affect confidentiality too.

QUICK DISTINCTION Strong cryptography can be bypassed when authenticators, endpoints, trust configuration, or recovery systems are compromised.

3.8–3.9 Site and Facility Security

Site Selection and Facility Design

Easy definition: Site security begins by evaluating natural, human, utility, transportation, environmental, jurisdictional, and neighborhood risks before construction or occupancy.

When it matters: Data centers, offices, evidence storage, critical facilities, alternate sites, and cloud/provider due diligence.

THINK LIKE A MANAGER Avoid high-consequence hazards where feasible, then layer deterrence, delay, detection, response, resilience, and recovery.

EXAM TRAPS

- Starting with locks and cameras before evaluating location risk.
- Placing primary and recovery sites in the same hazard or utility region.
- Ignoring emergency services, roads, flight paths, flood plains, civil unrest, and adjacent tenants.
- Assuming the landlord or provider addresses customer-specific risk.

QUICK DISTINCTION Site selection avoids or reduces inherent risk; facility controls mitigate risk that remains.

Physical Perimeters and Restricted Areas

Easy definition: Physical security uses layered zones, barriers, lighting, locks, guards, sensors, surveillance, visitor controls, and access records to protect people and assets.

When it matters: Perimeters, offices, data centers, wiring closets, media rooms, loading areas, and evidence storage.

THINK LIKE A MANAGER Design layers that detect and delay long enough for an effective response; enforce least privilege and accountability inside the perimeter.

EXAM TRAPS

- Treating cameras as preventive controls when they primarily detect or deter.
- Allowing tailgating through convenience entrances.
- Using one badge zone for all internal areas.
- Ignoring loading docks, roofs, utility entries, and shared walls.
- Storing evidence without restricted access and documented custody.

QUICK DISTINCTION Deterrent discourages; preventive blocks; detective identifies; delay buys response time. One control may serve multiple functions.

Power, HVAC, Fire, Water, and Environmental Controls

Easy definition: Facilities require stable power, cooling, humidity, fire detection/suppression, water protection, grounding, and monitoring to preserve safety and availability.

When it matters: Data centers, server rooms, wiring closets, industrial sites, archives, and recovery facilities.

THINK LIKE A MANAGER Design capacity and redundancy from business requirements, eliminate common-mode dependencies, test failover, and protect people before equipment.

EXAM TRAPS

- Sizing UPS as long-term generation; it usually bridges to generator or safe shutdown.
- Providing redundant equipment on one electrical or cooling path.
- Choosing fire suppression based only on equipment protection.
- Placing water pipes above critical equipment.
- Ignoring fuel contracts, maintenance, and load testing.

QUICK DISTINCTION UPS supplies immediate short-duration power; generators provide longer duration; redundancy works only when dependencies and transfer mechanisms are also resilient.

Fire Classes and Typical Suppression

Class	Fuel	Manager-oriented note
A	Ordinary combustibles	Water or suitable agents; protect people and contents.
B	Flammable liquids/gases	Smothering or chemical agents; do not spread burning liquid.
C	Energized electrical equipment	Use nonconductive agent; after de-energizing, classify by remaining fuel.
D	Combustible metals	Use specialized dry powder; ordinary agents may worsen fire.
K	Cooking oils/fats	Wet chemical systems for commercial cooking hazards.

3.10 Information-System Lifecycle

Stakeholder Needs and Requirements Analysis

Easy definition: The lifecycle begins by identifying stakeholders, mission, constraints, data, threats, obligations, and measurable security and assurance requirements.

When it matters: New systems, acquisitions, major changes, modernization, cloud migration, and replacement.

THINK LIKE A MANAGER Resolve needs and risk early; defects in requirements propagate through every later phase and are expensive to compensate for.

EXAM TRAPS

- Beginning architecture before scope and stakeholders are known.
- Letting security requirements remain vague or untestable.
- Ignoring operations, users, privacy, safety, support, and retirement stakeholders.
- Treating compliance requirements as the complete security baseline.

QUICK DISTINCTION Validation asks whether the right system is being built; verification asks whether the system was built according to specification.

Architecture, Development, Integration, and V&V

Easy definition: Architecture allocates requirements and trust; development implements the design; integration connects components; verification and validation provide evidence that requirements and stakeholder needs are satisfied.

When it matters: Design reviews, secure engineering, testing, acceptance, supply-chain integration, and authorization.

THINK LIKE A MANAGER Maintain traceability from requirement to design, control, implementation, test, finding, and risk decision.

EXAM TRAPS

- Testing only individual components and ignoring emergent integration risk.
- Confusing passing functional tests with security assurance.
- Allowing the development team to be the only assessor for high-risk systems.
- Accepting failed requirements without documented owner decision.

QUICK DISTINCTION Verification: built right against specifications. Validation: built the right system for intended use.

Deployment, Operations, Sustainment, and Change

Easy definition: Secure transition establishes approved configuration, data migration, training, monitoring, support, recovery, and ownership; operations maintain the system under controlled change.

When it matters: Production acceptance, baselines, patching, incidents, capacity, drift, vendor support, and continuous monitoring.

THINK LIKE A MANAGER Authorize a known baseline, monitor effectiveness and risk, control changes, and keep

documentation, recovery, and skills current.

EXAM TRAPS

- Deploying before operations accepts ownership and support.
- Treating security authorization as permanent.
- Allowing emergency changes to bypass later review and documentation.
- Ignoring configuration drift and expiring dependencies.
- Measuring availability without recoverability and business outcomes.

QUICK DISTINCTION Configuration management controls the approved state; change management governs proposed modifications and their risk.

Retirement and Disposal

Easy definition: Retirement removes a system from service while preserving required records, transferring functions and data, revoking access and trust, sanitizing assets, and updating dependencies and inventories.

When it matters: Decommissioning, migration, contract exit, cloud shutdown, hardware disposal, and mergers.

THINK LIKE A MANAGER Plan retirement during design, confirm business and legal requirements, migrate and validate, revoke identities and certificates, sanitize, and preserve evidence of completion.

EXAM TRAPS

- Turning off a server and calling it retired.
- Deleting cloud resources without exporting required records or keys.
- Leaving DNS, accounts, firewall rules, certificates, licenses, and monitoring references active.
- Sanitizing before confirming retention and legal holds.
- Ignoring downstream systems that depend on the retired service.

QUICK DISTINCTION End of operation is not end of accountability; retirement closes data, identity, dependency, contract, and asset lifecycles.

Cross-Domain Exam Traps for Domain 3

Choose the Level of Answer

Question focus	Best-answer level	Attractive trap
Business need or new system	Requirements, stakeholders, risk, and policy	Naming a product or algorithm first
Architecture weakness	Design principle, trust boundary, or layered control	Patching one symptom only
Crypto selection	Objective, approved standard, key lifecycle, implementation	Choosing the longest key or newest algorithm
OT/critical system	Safety, process impact, tested change, compensating controls	Applying enterprise IT action immediately
Facility resilience	Hazard analysis, dependencies, life safety, layered design	Buying a UPS or camera first
System acceptance	Traceable verification, validation, residual risk approval	Assuming vendor certification is sufficient

What Happens First?

- New architecture: identify stakeholders, mission, assets, data, obligations, threats, and measurable requirements.
- Control selection: determine risk and required security property before choosing technology.
- Cryptographic design: define objective, data lifetime, parties, trust, performance, and key lifecycle.
- Legacy or OT vulnerability: understand physical process and safety; coordinate and test before disruptive action.
- Facility project: assess site hazards and business continuity requirements before selecting individual controls.
- Cloud or third-party service: map shared responsibilities and trust boundaries before deployment.
- Retirement: confirm data migration, retention, dependencies, authorization, and legal holds before destruction.

ROOT-CAUSE TRAP When a system has repeated vulnerabilities, the best answer may be an architecture or lifecycle correction—secure design, reduced attack surface, isolation, requirements traceability, or replacement—not another compensating product.

Domain 3 Rapid Review

ONE-SENTENCE DOMAIN SUMMARY Translate business and risk requirements into simple, layered, verifiable architectures; protect trust and keys; account for platform, physical, and lifecycle realities; and maintain evidence that the system remains fit for its mission.

High-Yield Distinctions

Concept A	Concept B	Fast distinction
Bell–LaPadula	Biba	Confidentiality: no read up/write down vs. integrity: no read down/write up
Clark–Wilson	Brewer–Nash	Commercial transaction integrity vs. dynamic conflict-of-interest confidentiality
Model	Architecture	Formal policy rules vs. structural realization
Trust	Assurance	Reliance vs. evidence-based confidence
TCB	Security kernel	All trusted enforcement components vs. core reference-monitor implementation
TPM	HSM	Platform-bound trust/keys vs. specialized higher-assurance crypto/key device
VM	Container	Separate guest kernel vs. shared host kernel
Symmetric	Asymmetric	Shared secret and speed vs. key pair, distribution, signatures
Hash	MAC	Unkeyed digest vs. shared-secret integrity/authenticity
Encryption	Digital signature	Confidentiality vs. integrity/origin/nonrepudiation support
Verification	Validation	Built to specification vs. meets intended stakeholder need
Fail secure	Fail safe	Preserves security objective vs. prioritizes life safety
SASE	SSE	Networking plus security vs. security-services subset

Final Manager-Mindset Checklist

- What stakeholder, mission, safety, data, legal, and risk requirements drive the design?
- Which security property and threat is the control intended to address?
- Are trust boundaries, privileged components, dependencies, and failure modes explicit?
- Does the design use least privilege, secure defaults, complete mediation, simplicity, and layered protection?
- Is assurance based on relevant, current, scoped evidence?
- For cryptography, are purpose, algorithm, mode, protocol, key lifecycle, implementation, and agility addressed?
- Do platform and deployment assumptions match cloud, OT, IoT, containers, or other system realities?
- Are life safety, facilities, power, cooling, fire, physical access, and recovery included?
- Can each requirement be traced through design, implementation, verification, validation, and owner acceptance?
- Is secure transition, sustainment, change, retirement, and disposal planned?

Source and Scope Note

Coverage is aligned to the ISC2 CISSP Certification Exam Outline effective April 15, 2024. This independent study aid uses original explanations and does not reproduce exam questions or official courseware. Official outline: <https://www.isc2.org/certifications/cissp/cissp-certification-exam-outline>